Building our corpus with Marin's data-engineering approach

Source registry, global cleanup, topic-quality buckets, exposure-matched proxies

> ⓘ **Note**
>
> The code-level replication manual adds exact artifact schemas, dedup/decontamination constants, scorer-call-path differences and the current three-vector hero schedule. The cyber extension is a separate data experiment. In particular, the current token-only reference scorer and the separate type-conditioned fusion/bucketing path must not be conflated.

What is released, and what we actually downloaded

Your team's description is substantially correct. Marin's September 2026 pipeline report releases the source registry and processing code, not a duplicate hosting of every prepared upstream corpus. The pipeline's final training-store paths can point to Marin infrastructure. Possessing the code or a path string does not confer access to those bytes.

We downloaded the released mixture records and audited the actual Parquet files at Hub revision `7aa59d2158a6299bde95d24dd1b1b9c98f6e4d5a`:

Artifact	Verified local contents	Meaning
`harrier_18t75_d768.parquet`	116 rows; 200 phase-0 weight entries per inspected row	Experiments, not documents
Training modes in those 116 rows	72 `fresh_full`; 44 `resumed_cooldown`	Cooldown branches share earlier training; they are not 116 independent full trainings
Phase counts	34 one-phase; 82 two-phase	Two-phase scheduling exists in the records; it is not universally necessary
`full_scale_token_count`	18.75T for every row	Target-scale exposure reference; not tokens stored inside the Parquet
Two-phase exposure fields	15T phase 0; 3.75T phase 1	80/20 target-scale phases; proxy costs must come from actual training configs
Tokenizer	Marin tokenizer pinned at `a5ca45f2feb6c959bd87b81689aa7279b5bdcaa2`	Different from our earlier OLMoE tokenizer candidate
`swarm_runs.parquet`	840 rows at this snapshot; inspected first weight vector has 168 entries	Different record schema; do not assume all exports have 200 cells

The report describes 872 observations in its particular analysis and over 1,000 small experiments overall. The present files and that historical analysis are different snapshots/scopes. **Do not silently equate 116, 840, 872, or 1,000, or add them together.** A newer registry coexists with legacy subsets. The audit, dataset card, pinned Parquet files and `scripts/audit_marin_proxy_records.py` preserve our evidence.

Each legacy row includes source-store identity, tokenizer, phase weights, allocated tokens, per-cell epochs, cumulative epochs, evaluation paths, grouped BPB and target/guardrail scores. These fields let us audit exposure and scoring; they are not enough to reconstruct unavailable text or every training artifact. A low macro-BPB row alone does not identify a causal best mixture: phase history, availability, task coverage and selection differ.

Version discipline: historical report versus present implementation

The pipeline report links Marin commit `64a9483cf1a928e6db97b83130d8a5d171a4aead`; selected files from that version are saved in `references/marin/2026-marin-data-pipeline-overview/pinned-code/`. Our ignored checkout is at `1d274b52c636b6f2a9177b1d4c14b2f74795a8a6`.

The report describes Harrier embeddings, 40 topics and 5 quality bands. The present reference DAG has evolved: its README describes Luxical embeddings, a pooled fast-transformer scorer, refined fuzzy containment checks, configurable cluster views and exemptions. Follow one coherent pinned version. Mixing a historical classifier, a new bucket definition and old proxy weights does not reproduce Marin.

Useful code locations, relative to `tmp/marin/`:

Component	Location to inspect	Why it matters
Source registry	`lib/marin/src/marin/datakit/sources.py`	Named components, download/normalize chains, approximate upstream-token counts
Canonical normalization	`lib/marin/src/marin/datakit/normalize.py`	Stable document identities and metadata
Reference DAG	`experiments/datakit/reference_pipeline.py`	Dependencies, hashed artifacts, global joins, resources and CLI
Exact dedup	`experiments/datakit/global_exact_dedup.py`	Cross-source canonical representatives
Fuzzy cleanup	`experiments/datakit/scripts/fuzzy_cluster_text.py`, `fuzzy_cluster_verify.py`	Candidate search versus verified duplicate removal
Eval corpus	`experiments/datakit/decontam/prepare_eval_corpus.py`	The exact material excluded from training
Scoring and calibration	`experiments/datakit/cluster/quality/fast_transformer/`	Quality model, content-type calibration and bucket rules
Final store	`experiments/datakit/store/`	Joins attributes, removes marked records, routes tokens to cells
Pilot/testbed	`experiments/datakit/testbed/`, `docs/design/datakit_testbed.md`	Small real-data validation before full jobs

Stable source links can be constructed with the pinned commit in our source pins. Paths and GCS prefixes in experiments are references, not portable storage configuration.

What the historical pipeline measured

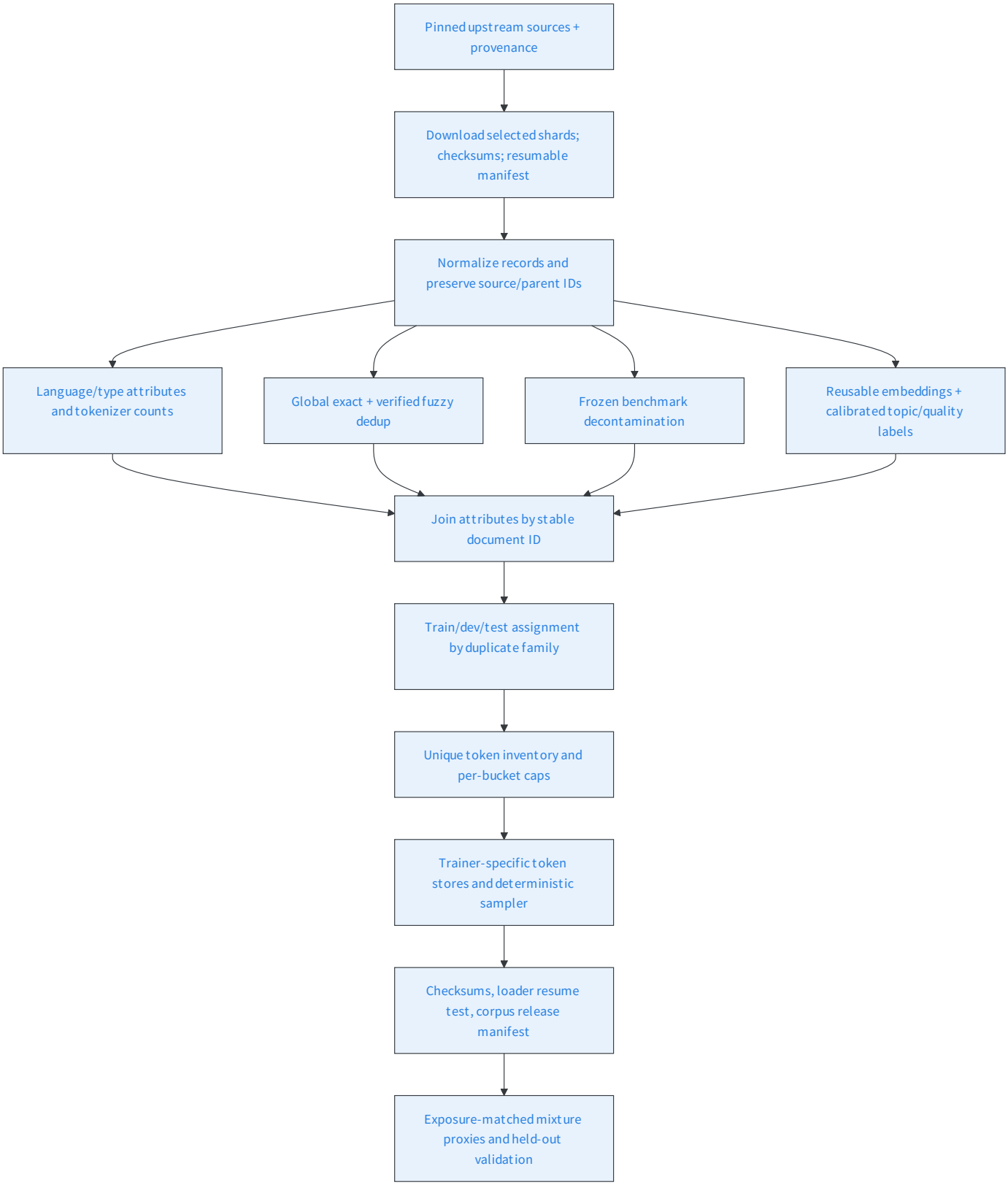
The report starts from 152 source groups, 18.71B documents and 25.25T **Llama-3** tokens. It reports removing 2.33B duplicate documents / 2.13T tokens and another 13.66B tokens under its decontamination policy. These are historical corpus measurements. They are not a forecast that our subset will retain exactly the same fraction.

Exact dedup uses normalized hashes across sources. Fuzzy candidate search uses MinHash on word 5-grams plus LSH, with additional verification needed to avoid deleting distinct DNA, tool calls or other templated data. Benchmark decontamination checks exact 13-word n-grams: if at least 50% of eligible occurrences in any paragraph overlap the eval material, the document is flagged. This detects substantial verbatim overlap; paraphrases and contamination not represented in that eval corpus can remain.

The historical topic procedure samples Harrier embeddings, makes 5,000 fine clusters, then merges them into 40 broader topics. A separately trained quality model supplies five content-calibrated bands. **Quality labels are sampling controls, not an instruction to delete every low-quality bucket.** The useful quality/diversity tradeoff depends on budget, repetition and task objectives.

In the present README, fuzzy verification compares retained longer representatives with candidates using distinct word 3-gram containment at 75%, with search limits and source exemptions. The defaults affect results; copy them only after inspecting errors on our data. This is a changed implementation, not a contradiction of the report's high-level method.

Our release pipeline and required outputs



This is a proposed portable implementation of the principles. It does not claim that Marin's infrastructure runs unmodified on our cluster.

1. Choose a manageable source set

Start with the sources already accessible to the team: selected Dolma components, original Nemotron-CC quality partitions, and explicitly chosen math/code sources. Record the **exact Dolma release and component**, not just "Dolma." Several registry sources come from later releases and require separate access or transformations. Original Nemotron-CC

and Nemotron-CC v2/v2.1 are different corpora; do not mix their partition names or assume identical accessibility.

Our existing public CC index has 31,279 compressed objects partitioned by quality, actual/synthetic kind and transformation subtype. Only its index/pilot URL manifests have been fetched into this repo. `scripts/select_cc_shards.py` can select reproducible shard samples without downloading the whole corpus. Preserve partition metadata from the path because individual records need not carry those labels.

Do **not** remove a presumed 15% multilingual share without measuring it. A dataset name, a translated document, a code comment and a mixed-language page are different cases. Label language at document/paragraph level, retain code/math exemptions where appropriate, and compare English-only versus limited multilingual coverage if that choice affects the product. A removed-document fraction is not necessarily a removed-token fraction.

2. Normalize once and retain lineage

A practical neutral record is:

```
{
  "doc_id": "stable hash under normalization-v1",
  "text": "...",
  "source_id": "nemotron-cc/quality=high/kind=actual",
  "source_revision": "pinned manifest hash",
  "upstream_record_id": "warc identifier or source row",
  "parent_ids": [],
  "license_record": "source registry entry",
  "language": "en",
  "content_type": "prose",
  "normalization_version": "v1"
}
```

Use partitioned Parquet for canonical text and separate keyed attributes for dedup, scores and tokens. Empty records and whitespace rules need measured counters. Preserve original identifiers and synthetic parent links. Changing normalization changes hashes, dedup and downstream artifact identities; it requires a new release.

3. Deduplicate the combined candidate pool

Per-source dedup alone misses overlap between Dolma, Nemotron, FineWeb and their descendants. Build a cross-source exact index, then fuzzy candidates and verified containment. Select representatives with a deterministic, documented priority; retain links from removed records to their representative. Audit false positives by prose, code, mathematics and short/templated documents, especially cybersecurity logs and command transcripts where a small edit may be meaningful.

Do this **before final unique-token accounting**. Keep a stratified human review sample from retained and removed pairs. Report exact removals, verified fuzzy removals, exemptions and source-pair overlap separately. Synthetic parent grouping is an additional lineage control; MinHash does not establish semantic independence.

4. Freeze evaluation exclusions before training

Prepare the fixed target/guardrail evaluation corpus, normalize it consistently, build matching signatures and remove flagged training documents. Save benchmark revision, split, prompt/answer extraction policy, n-gram length, threshold and exclusions hash. Inspect flags on short text and common boilerplate. Reserve an internal holdout beyond public benchmark exclusion; public benchmarks alone cannot protect against tuning leakage.

For cyber-specific synthetic generation, deduplicate prompts, source documents and outputs against the same exclusions. Track generator revision, template, seed and parent record. Split document/derivation families before assigning train versus test. Evaluate on independently sourced tasks; generated exercises derived from evaluation material cannot serve as independent evidence.

5. Keep topic and quality as reusable attributes

A full 40×5 taxonomy is optional for the first release. Begin with source × quality × broad domain metadata that already exists, then decide whether the gain from Harrier/Luxical embeddings and custom scoring warrants its cost. This reduced plan is **not an exact Marin reproduction**.

For a faithful historical reproduction, recover the exact Harrier checkpoint/instruction/truncation, centroid sample, clustering/merge seed, quality-model checkpoint, rubric and content-type calibration. A code path accepting an internal checkpoint directory is not evidence that the checkpoint bytes are publicly obtainable. Until those inputs are resolved, list them as missing dependencies rather than claiming 200 equivalent buckets.

Quality scoring requires calibration across prose, code and math. Audit score distributions and examples per source/type. Reusing an upstream “high” label is a reasonable baseline, but it is not numerically equivalent to Marin’s q4.

6. Tokenize and export for the selected trainer

Freeze tokenizer repository/revision, vocabulary padding, Unicode handling, EOS policy, document boundaries, loss mask and context packing. Recount every source and bucket under that tokenizer. Marin’s reported Llama-3 counts and the legacy experiments’ Marin-tokenizer counts cannot be substituted for our OLMoE-tokenizer counts.

Maintain a neutral document/attribute release and create either Megatron indexed `.bin/.idx` stores or a TorchTitan Grain-compatible token source. A Levanter store is not automatically either format. Keep source/cell IDs and document offsets available to audit sampling, repetition and decontamination. Validate loss masks and boundary handling on short handcrafted records before bulk export.

7. Learn mixtures while preserving hero exposure

For hero bucket availability U_b and exposures D_b, set proxy exposures $D'_b = sD_b$ and proxy pool $U'_b = sU_b$. Then

$$\frac{D'_b}{U'_b} = \frac{D_b}{U_b}.$$

The scale factor s follows the proxy/hero token budget; do not make a small proxy see the full high-quality pool while the hero must repeat it several times. That tests a different repetition regime. At very small scales, tiny buckets need minimum sample sizes and repeated draws must be logged.

Run a proportional capped baseline, the proposed mixture, quality-tilted variants, code/math variants and a low-quality inclusion ablation. For the final phase, compare fixed-mixture and quality/domain cooldown branches from the **same prefix**, keeping decay identical. Also run fresh full-training finalists to test whether the conditional cooldown result survives the whole trajectory.

Fit a constrained surrogate on token weights, availability, phases and repetition. Weights sum to one, so a bucket-performance correlation is not an isolated treatment effect. Hold out whole runs/prefix families; randomly splitting sibling cooldown branches leaks shared history. Optimize within observed support and epoch caps, then validate the proposed mixture in fresh runs at a larger proxy. The September report itself notes that proxy-to-larger-model transfer remains to be tested.

Size the effort before promising a date

Measure a stratified pilot of roughly 1M documents across every selected source/quality/type, then a 10B-token end-to-end integration release. These are our proposed engineering milestones, not Marin’s published timing guarantees. Deliver retained-token estimates with uncertainty, measured source access speed, stage throughput, temporary storage and sample audits.

For stage j, estimate wall time using **measured aggregate throughput** v_j and input volume X_j:

$$t_j \approx X_j / v_j.$$

Include retries, skewed dedup groups, stage dependencies and bottleneck storage. For GPU annotation, measure documents/s and tokens/s under the actual length distribution; a 0.6B embedding model's name alone is not a cost estimate. Charge GPUs borrowed from the 96-GPU allocation to its budget.

Useful storage bounds:

Artifact	Approximate payload, before compression/overhead
D stored tokens	$2D$ bytes for a suitable uint16 vocabulary, $4D$ for uint32
3T stored tokens	6TB or 12TB respectively
1B document embeddings, 1,024 dimensions BF16	2.048TB
10M such embeddings	20.48GB
Text, MinHash, sort/shuffle spill, offsets, replicated caches	Additional; measure rather than assuming token files dominate

Stored unique tokens and repeated training exposures are different storage quantities: repeated sampling need not duplicate the token files. Token IDs must fit the chosen dtype; vocabulary padding and sentinel values also matter.

People/work streams: one owner for source access and lineage; one for distributed cleanup and stores; one for annotation/mixture experiments; one for trainer/eval integration. These are work responsibilities, not an asserted staffing minimum. A single person can combine roles, with a corresponding critical path.

Release acceptance

A corpus release is ready when source checksums and access are verified; canonical counts reconcile through each removal stage; cross-source duplicate-family splits are disjoint; benchmark exclusions are versioned; quality/language samples pass review; each bucket has unique-token availability and repetition caps; and both the sampler and its resumed state reproduce the intended token fractions. Keep one immutable release manifest linking all artifacts.

The next decision is therefore concrete: **which selected sources survive the pilot, and how many usable unique tokens do they yield per bucket?** The 96-GPU guide translates that inventory into exposure requirements and launch conditions.